reference instead. In this case it's undefined behavior to call the lambda once the variables it references have been destroyed by exiting the function or block scope to which they belong, just as it's undefined behavior to reference a variable that has already been destroyed in any other circumstance.

A lambda function that captures all the local variables by reference is introduced using `[&]`, as in the following example:

```
int main()
{
    int offset=42;                                                    ❶
    std::function<int(int)> offset_a=[&](int j){return offset+j;};    ❷
    offset=123;                                                       ❸
    std::function<int(int)> offset_b=[&](int j){return offset+j;};    ❹
    std::cout<<offset_a(12)<<","<<offset_b(12)<<std::endl;            ❺
    offset=99;                                                        ❻
    std::cout<<offset_a(12)<<","<<offset_b(12)<<std::endl;            ❼
}
```

Whereas in the `make_offseter` function from the previous example we used the `[=]` lambda introducer to capture a copy of the offset, the `offset_a` function in this example uses the `[&]` lambda introducer to capture `offset` by reference ❷. It doesn't matter that the initial value of `offset` is 42 ❶; the result of calling `offset_a(12)` will always depend on the current value of `offset`. Even though the value of `offset` is then changed to 123 ❸ before we produce the second (identical) lambda function `offset_b` ❹, this second lambda again captures by reference, so the result depends on the current value of `offset`.

Now, when we print the first line of output ❺, `offset` is still 123, so the output is `135,135`. However, at the second line of output ❼, `offset` has been changed to 99 ❻, so this time the output is `111,111`. Both `offset_a` and `offset_b` add the current value of `offset` (99) to the supplied argument (12).

Now, C++ being C++, you're not stuck with these all-or-nothing options; you can choose to capture some variables by copy and some by reference, and you can choose to capture only those variables you have explicitly chosen just by tweaking the lambda introducer. If you wish to *copy* all the used